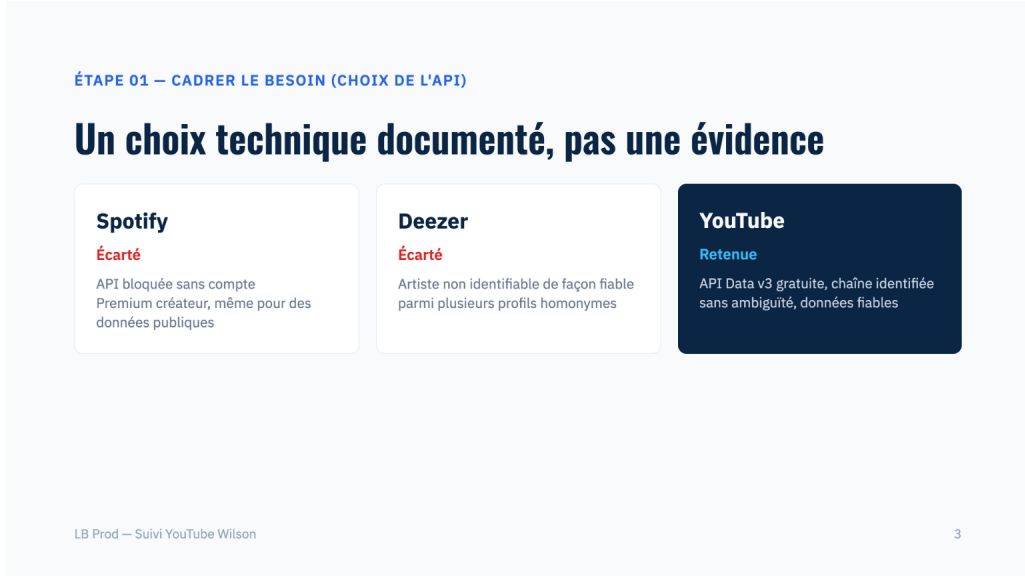


Bonjour, je suis Loïc. Je vous présente le pipeline que j'ai construit pour suivre la performance YouTube de l'artiste Wilson et aider LB Prod à décider où investir en promotion.



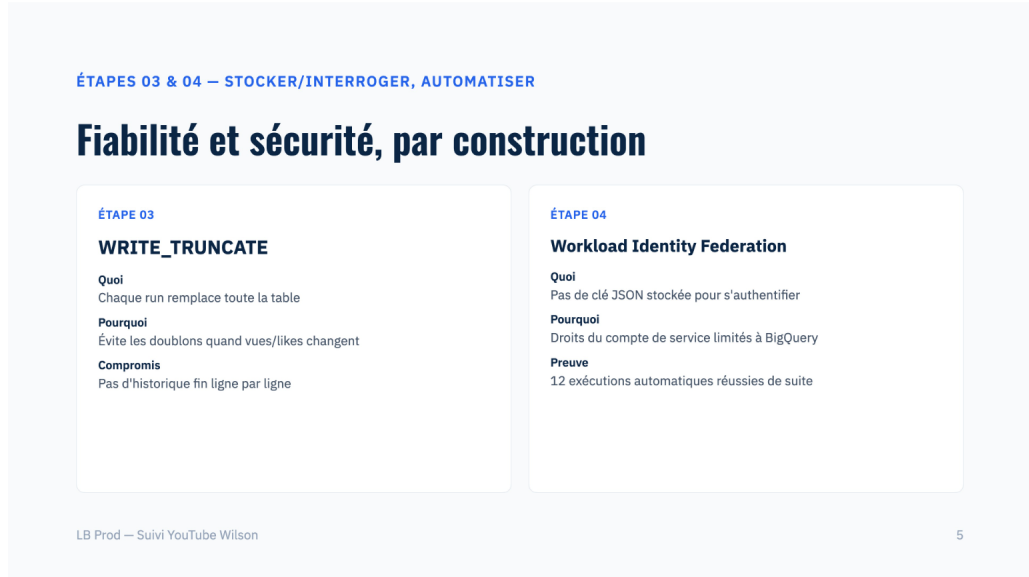
LB Prod est un label musical fictif qui produit plusieurs artistes, dont Wilson. Le constat de départ : la direction dépense un budget de promotion sans savoir précisément quels formats de vidéos fonctionnent le mieux pour lui — clip officiel, lyrics vidéo, live. Deux questions guident le projet. D'abord, quels formats génèrent le plus de vues et le plus d'engagement, pour savoir où réinvestir. Ensuite, comment suivre ça dans la durée sans ressaisir des chiffres à la main chaque semaine. L'objectif du pipeline que je vais vous montrer est justement de répondre à ces deux questions à partir de données réelles, collectées automatiquement.



Avant de choisir YouTube, j'ai testé deux autres pistes. Spotify d'abord : son API bloque l'accès aux statistiques, même publiques, tant que le compte créateur de l'application n'est pas en Premium — donc inutilisable ici. Deezer ensuite : impossible d'identifier Wilson de façon fiable, plusieurs artistes homonymes existent sur la plateforme, sans identifiant unique garanti. J'ai donc retenu YouTube Data API v3 : elle est gratuite, bien documentée, la chaîne de Wilson est identifiée sans ambiguïté par son ID, et elle donne accès aux vues, likes et commentaires par vidéo — exactement ce qu'il fallait pour répondre à la question métier.



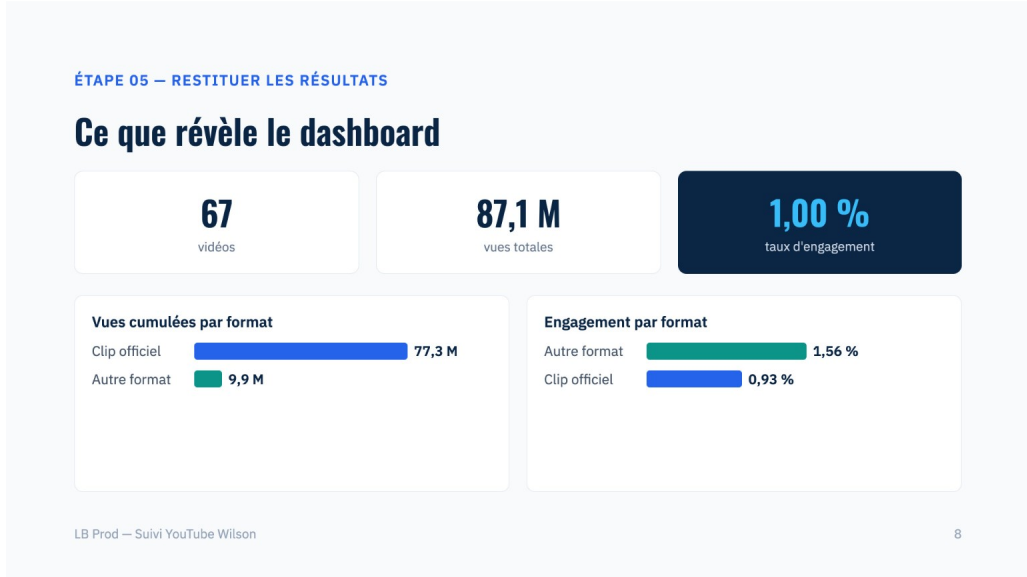
Voici l'architecture d'ensemble, en quatre étapes. L'API YouTube fournit les données brutes sur chaque vidéo de la chaîne. Un script Python les collecte : il contrôle que chaque appel API a bien réussi avant de continuer, et il nettoie les données — conversion des textes en nombres, suppression des doublons, horodatage de chaque collecte. BigQuery stocke ensuite ces données dans une table interrogeable en SQL. Et Looker Studio se connecte directement à BigQuery pour restituer tout ça sous forme de dashboard, avec des KPI et des graphiques mis à jour automatiquement.



Deux décisions techniques que je veux assumer clairement. La première : le chargement dans BigQuery utilise WRITE_TRUNCATE, c'est-à-dire que chaque exécution remplace entièrement la table plutôt que d'ajouter des lignes. Pourquoi ce choix : les vues et les likes évoluent sur les mêmes vidéos, donc un ajout créerait des doublons. Le compromis assumé, c'est qu'on n'a pas d'historique fin ligne par ligne — on garde juste l'état le plus récent. La seconde décision concerne l'automatisation : GitHub Actions s'authentifie auprès de Google Cloud via Workload Identity Federation, et non avec une clé JSON stockée dans le dépôt. Ça veut dire aucun secret longue durée à sécuriser, et le compte de service n'a de droits que sur BigQuery, rien d'autre. Preuve que ça fonctionne : douze exécutions automatiques consécutives, avec les logs archivés sur GitHub.



Je reviens sur les slides. Voyons maintenant ce que ces données révèlent, chiffres à l'appui.



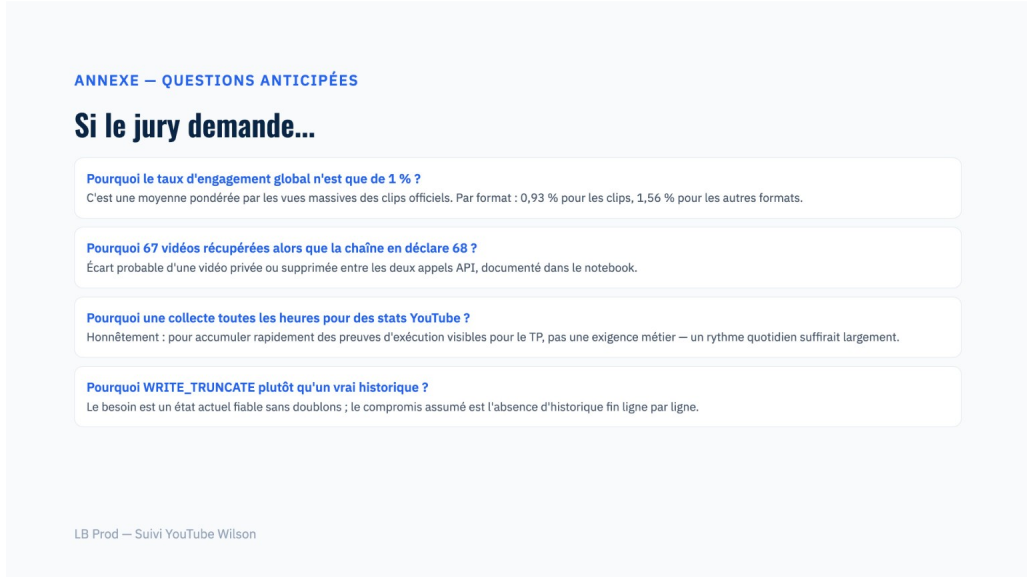
67 vidéos, 87,1 millions de vues, 1 % d'engagement moyen. Les clips officiels font 88 % des vues, mais ce sont les autres formats qui engagent le mieux : 1,56 % contre 0,93 %.



Conclusion : priorité budgétaire aux clips officiels pour la portée, et on exploite les autres formats pour fidéliser l'audience déjà acquise. Tout le code est public sur GitHub.



Merci pour votre attention, je suis prêt à répondre à vos questions.



Slide de réserve, pas montrée dans le déroulé normal — à utiliser seulement si une de ces questions tombe pendant les 4 minutes de Q&A.



Place à la démo : notebook, BigQuery, GitHub Actions, puis le dashboard. Les onglets sont déjà ouverts dans l'ordre pour basculer vite.